

Tema 5: Pipes, FIFOs y Senales

Este tema cubre la comunicacion entre procesos y las senales. Las preguntas sobre pipes representan alrededor del diez por ciento del examen, y hay trampas classicas sobre deadlocks y SIGPIPE que debes conocer.

Que es un pipe

Un pipe, o tuberia, es un mecanismo de comunicacion entre procesos. Piensa en el como un tubo: un proceso escribe por un extremo y otro proceso lee por el otro. Es unidireccional, o simplex, lo que significa que los datos solo fluyen en una direccion. Internamente, tiene un buffer limitado, que en Linux es tipicamente de sesenta y cinco kilobytes.

Un pipe tiene dos extremos, que se representan como dos descriptores de fichero en un array de dos posiciones. La posicion cero es el extremo de lectura, y la posicion uno es el extremo de escritura. Recuerda: cero es leer, uno es escribir.

La llamada pipe

La llamada pipe recibe un array de dos enteros y lo rellena con los dos descriptores del pipe. Devuelve cero si tiene exito, o menos uno si hay error. Es fundamental llamar a pipe antes de fork, para que padre e hijo hereden los dos extremos del pipe y puedan comunicarse.

Despues del fork, cada proceso debe cerrar el extremo que no usa. Si el hijo es el lector, cierra fd sub uno, que es el de escritura. Si el padre es el escritor, cierra fd sub cero, que es el de lectura. Cerrar los extremos no usados es obligatorio para que el pipe funcione correctamente.

Reglas de comportamiento del pipe

Estas reglas son preguntas classicas de examen. Si intentas leer de un pipe vacio y alguien tiene abierto el extremo de escritura, el read se bloquea esperando datos. Si intentas escribir en un pipe lleno, el write se bloquea esperando que alguien lea.

Ahora bien, si lees de un pipe vacio y nadie tiene abierto el extremo de escritura, read devuelve cero bytes, que significa fin de fichero. Esto es una condicion normal, no un error. Y si escribes en un pipe y nadie tiene abierto el extremo de lectura, el proceso recibe la senal SIGPIPE, que por defecto lo termina.

Hay una situacion de deadlock o interbloqueo muy importante. Si un unico proceso intenta escribir un bloque grande en un pipe y luego leer, puede quedarse bloqueado para siempre. Esto ocurre porque el buffer del pipe es limitado. Si intentas escribir treinta y dos megabytes, write llena el buffer de sesenta y cinco kilobytes y se bloquea esperando que alguien lea. Pero el read que viene despues nunca se ejecuta porque write esta bloqueado. Es un deadlock.

Ejemplo de examen: tienes un array de treinta y dos megabytes, haces pipe, y luego write de todo el array seguido de read. La respuesta es que se queda bloqueado para siempre.

Pipes en el shell

Cuando escribes en el shell algo como `cat archivo | grep palabra | wc -l`, el shell crea pipes entre los procesos. La salida estandar de `cat` se conecta a la entrada estandar de `grep`, y la salida de `grep` se conecta a la entrada de `wc`. Internamente, el shell usa `pipe`, `fork`, `dup2` y `exec` para conseguir esto.

Ejemplo practico de pipe con fork

El patron tipico es: crear el pipe con `pipe`, hacer `fork`, el hijo cierra el extremo de escritura y lee del pipe, el padre cierra el extremo de lectura y escribe en el pipe, y el padre hace `wait` para esperar al hijo. Si usas `dup2` para redirigir `stdin` o `stdout` a los extremos del pipe, puedes encadenar comandos como `ps aux | wc -l`.

FIFOs o Named Pipes

Un FIFO es un pipe con nombre en el sistema de ficheros. Se crea con el comando `mkfifo` en el shell o con la funcion `mkfifo` en C. La diferencia con un pipe normal es que un FIFO tiene una ruta y puede ser abierto por procesos que no tienen relacion de parentesco.

El comportamiento es similar al pipe, pero con particularidades en la apertura. Si abres un FIFO en modo solo lectura, se bloquea hasta que alguien lo abra para escritura. Y viceversa.

Senales

Una senal es un mecanismo de notificacion asincrona a un proceso. Interrumpe la ejecucion normal del proceso y puede tomar tres acciones: la accion por defecto, que normalmente es terminar el proceso; ignorar la senal; o ejecutar un manejador personalizado.

Las senales se clasifican en sincronas, que son consecuencia de la ejecucion del proceso, como `SIGSEGV` por violacion de segmentacion o `SIGFPE` por error aritmetico; y asincronas, que son enviadas por otros procesos o por eventos externos, como `SIGINT` que se envia con Control C o `SIGTERM` que es la senal de terminacion.

Las senales mas importantes

`SIGINT`, que es la senal numero dos, se envia con Control C y termina el proceso por defecto. `SIGQUIT`, numero tres, se envia con Control barra invertida y genera un core dump. `SIGKILL`, numero nueve, mata el proceso y no puede ser capturada ni ignorada. Esto es importantisimo: `SIGKILL` no se puede bloquear, capturar ni ignorar. Es la forma garantizada de matar un proceso.

`SIGSEGV`, numero once, indica una violacion de segmentacion. `SIGPIPE`, numero trece, se envia cuando escribes en un pipe sin lector. `SIGTERM`, numero quince, es la senal de terminacion software que si puede ser capturada. `SIGSTOP`, numero diecisiete, para el proceso y tampoco puede ser capturada. `SIGCHLD`, numero veinte, se envia al padre cuando un hijo cambia de estado.

Pregunta de examen: SIGKILL no se puede ignorar. SIGSTOP tampoco. Ctrl C envia SIGINT, no SIGKILL. Ctrl Z envia SIGTSTP, que para el proceso.

Manejar senales con signal

La funcion signal registra un manejador para una senal. Recibe el numero de senal y un puntero a funcion que sera el manejador. El manejador es una funcion void que recibe un int como parametro, que es el numero de senal recibida. Tambien puedes pasar SIG DFL para restaurar la accion por defecto, o SIG IGN para ignorar la senal.

Los manejadores de senales deben ser muy cuidadosos: deben ser reentrantes, no deben usar variables globales ni funciones no reentrantes como malloc o free, deben ser pequenos, y no deben modificar errno.

Bloqueo de senales

Con sigprocmask puedes bloquear senales temporalmente. Bloquear una senal es diferente de ignorarla. Si ignoras una senal, se descarta. Si la bloqueas, queda pendiente, y cuando la desbloqueas, se entrega. Esto permite proteger secciones criticas de codigo que no deben ser interrumpidas.

La mascara de senales de un proceso indica que senales tiene bloqueadas, no ignoradas. Esta es una pregunta de examen frecuente.

Alarmas

La funcion alarm programa un temporizador que envia SIGALRM al proceso despues de un numero de segundos. Si llamas a alarm de cero, cancelas la alarma pendiente. Si llamas a alarm dos veces, la segunda sobrescribe la primera. Se usa para implementar timeouts.

Job control, sesiones y demonios

Una sesion es un grupo de procesos con un lider. Un grupo de procesos es un conjunto de procesos relacionados. Un job del shell es un grupo de procesos. Solo un grupo puede estar en primer plano, y es el que recibe las senales del terminal como SIGINT con Control C. Si un proceso en segundo plano intenta leer del terminal, recibe SIGTTIN.

Un demonio es un proceso de servicio que se ejecuta sin terminal. Para crear un demonio, el proceso crea una nueva sesion con setsid, cambia al directorio raiz, y cierra o redirige los descriptores de entrada, salida y error. La funcion daemon de libc simplifica este proceso.

Para evitar zombies cuando no te interesa el resultado de los hijos, puedes hacer signal de SIGCHLD con SIG IGN. Esto le dice al kernel que limpie automaticamente los procesos hijos terminados.